

Documentazione: La Partita di Tris



Corso: Laboratorio di Sistemi Operativi @ Università degli studi di Napoli Federico II (UNINA)

Progetto: La Partita di Tris

Autori: Oreste Leone N86/1980 , Giuseppe Falso N86/2941

Indice dei Contenuti

1. **Panoramica del Progetto**
2. **Architettura del Sistema**
 - 2.1 Modello di Concorrenza (Server)
 - 2.2 Strutture Dati Condivise
3. **Protocollo di Comunicazione**
 - 3.1 Formato del Pacchetto
 - 3.2 Tipi di Messaggi Principali
4. **Dettagli Implementativi e Scelte Progettuali**
 - 4.1 Gestione del Matchmaking (Code FIFO)
 - 4.2 Sincronizzazione di Gioco
 - 4.3 Comunicazione Inter-Thread (Pipe)
 - 4.4 Gestione Disconnessioni
 - 4.5 Sistema di Replay (Riciclo)
5. **Dettagli Tecnici Client (CLI)**
6. **Conclusioni**

1. Panoramica del Progetto

Il progetto implementa una versione multiplayer del gioco del Tris basata su un'architettura **Client-Server** in linguaggio C, conforme agli standard POSIX. Il sistema è progettato per supportare molteplici connessioni concorrenti, permettendo agli utenti di incontrarsi in una lobby comune, creare nuove partite o unirsi a quelle esistenti.

La caratteristica distintiva dell'architettura è l'uso di un modello di concorrenza ibrido che combina il multithreading ("One Thread Per Client") con il multiplexing I/O (`select()`) per gestire in modo efficiente sia la comunicazione di rete che la comunicazione interna tra i thread del server.

La struttura del progetto è così definita:

```
/
├── Makefile
├── README.md
├── LeoneFalso-Tris-Doc.pdf
├── client/
│   └── client.c
├── common/include/
│   ├── datastructures.h
│   └── protocol.h
└── server/
    ├── server.c
    ├── thread_logic.c
    └── thread_logic.h
```

2. Architettura del Sistema

2.1 Modello di Concorrenza (Server)

La scelta architetturale principale ricade sul pattern **One Thread Per Client**.

- **Listener Principale:** Il thread principale accetta le nuove connessioni TCP e, per ogni client, avvia un thread dedicato (`client_thread_handler`).
- **Gestione I/O (Select):** Ogni thread worker non è bloccante su una singola operazione. Utilizza la system call `select()` per monitorare simultaneamente due descrittori di file:
 1. **Socket del Client (`client_sd`):** Per ricevere comandi di rete (es. mosse, chat, richieste).
 2. **Pipe Interna (`pipe_fd[0]`):** Un canale di comunicazione unidirezionale creato per ogni giocatore, utilizzato dagli altri thread per inviare notifiche asincrone (es. "Trovato avversario", "Aggiorna Lobby", "Disconnessione avversario").

Questa scelta permette di evitare l'uso di segnali UNIX complessi e garantisce che ogni client possa ricevere aggiornamenti immediati (come l'ingresso di un nuovo giocatore in lobby) anche mentre è bloccato in attesa di input di rete.

2.2 Strutture Dati Condivise

Lo stato globale del server è incapsulato in una struttura **ServerState**, protetta da meccanismi di mutua esclusione (**pthread_mutex_t**).

- **PlayerNode (Lista Giocatori)**: Una lista concatenata che mantiene lo stato di tutti i client connessi. Ogni nodo contiene il socket, lo stato corrente (**IN_LOBBY**, **IN_GAME**), la pipe di comunicazione interna e i dati della partita corrente.
- **GameNode (Lista Partite)**: Una lista concatenata delle partite. Ogni nodo rappresenta una stanza di gioco e contiene:
 - Stato (**WAITING**, **RUNNING**, **FINISHED**).
 - Riferimenti ai descrittori dei due giocatori (Owner e Opponent).
 - La griglia di gioco 3x3.
 - **Coda Sfidanti (ChallengerQueue)**: Una lista FIFO (**ChallengerNode**) specifica per ogni partita, che memorizza i giocatori in attesa di approvazione da parte dell'host.

L'uso di liste concatenate invece di array statici permette una gestione dinamica della memoria, scalando con il numero di utenti senza limiti predefiniti rigidi (se non quelli di sistema).

3. Protocollo di Comunicazione

La comunicazione tra Client e Server avviene tramite un protocollo binario custom su TCP. Questo garantisce efficienza e affidabilità rispetto a protocolli testuali.

3.1 Formato del Pacchetto

Ogni messaggio è composto da un Header fisso seguito da un Payload variabile:

```
typedef struct {
    MessageType type; // Enum (4 byte)
    int payload_size; // Dimensione dati successivi (4 byte)
} PacketHeader;
```

3.2 Tipi di Messaggi Principali

- **CMD_*** (Da Client a Server):
 - **CMD_CREATE_GAME**: Richiede la creazione di una nuova stanza.
 - **CMD_JOIN_GAME**: Richiede l'inserimento nella coda di una stanza.
 - **CMD_MOVE**: Invia le coordinate di una mossa (0-8).
 - **CMD_HOST_DECISION**: L'host accetta o rifiuta uno sfidante.
- **RSP_*** (Da Server a Client):
 - **RSP_GAME_LIST**: Invia la lista delle partite attive (per la lobby).
 - **RSP_LOBBY_UPDATE**: Segnale vuoto che forza il client a ricaricare la lista partite.
 - **RSP_GAME_OVER**: Notifica l'esito (Vittoria/Sconfitta/Pareggio/Ritiro).

4. Dettagli Implementativi e Scelte Progettuali

4.1 Gestione del Matchmaking (Code FIFO)

Abbiamo scelto di implementare un sistema di **matchmaking con approvazione**. Quando un giocatore vuole unirsi a una partita in stato **WAITING**:

1. Viene inserito in una coda FIFO (**ChallengerNode**) all'interno del **GameNode**.
2. Il server notifica l'Host *so/o* del primo sfidante in coda (**MSG_JOIN_REQUEST**).
3. L'Host ha potere decisionale:
 - **Accetta**: La partita inizia, la coda viene svuotata e gli altri sfidanti vengono respinti automaticamente.
 - **Rifiuta**: Lo sfidante viene rimosso e il server propone automaticamente il prossimo in coda.

Questa logica previene race condition dove più giocatori credono di essersi uniti alla stessa partita.

4.2 Sincronizzazione di Gioco

- **Validazione Server-Side**: Il server è l'unica autorità. Verifica turno, validità della cella e condizioni di vittoria (**check_win**, **check_draw**).
- **Update Client**: il client aggiorna la griglia locale in seguito alla ricezione della validazione e update dello stato dal server, sia in caso di proseguimento del match che in caso di errori come **RSP_INVALID_MOVE**.
- **Turni**: Gestiti tramite confronto del **socket descriptor** corrente.

4.3 Comunicazione Inter-Thread (Pipe)

Una delle sfide principali è stata permettere a un thread (es. gestore dell'Host) di comunicare con un altro (es. gestore dello Sfidante).

Soluzione: Ogni **PlayerNode** possiede una **pipe**.

- Il thread A scrive nella **pipe_fd[1]** del thread B.
- Il thread B, che è in **select()**, si sveglia rilevando attività su **pipe_fd[0]**, legge il messaggio interno (**InternalMessage**) e lo processa (es. inviando un pacchetto TCP al proprio client).

4.4 Gestione Disconnessioni

Il sistema è robusto contro le disconnessioni improvvise (**handle_disconnect**):

- **In Lobby**: Il giocatore viene rimosso e la lista aggiornata per tutti.
- **In Partita**:
 - Se esce l'Host: La partita viene distrutta, l'avversario vince a tavolino (**MSG_OPPONENT_QUIT**), e tutti gli sfidanti in coda vengono notificati del rifiuto.
 - Se esce l'Avversario: L'Host vince a tavolino e la partita viene marcata **FINISHED**.

4.5 Sistema di Replay (Riciclo)

Alla fine di una partita, il server gestisce la volontà di rigiocare (`CMD_PLAY_AGAIN`):

- **Vincitore:** Ha la priorità. Se vuole rigiocare, la stanza (`GameNode`) viene "riciclata" (stato reset a `WAITING`, griglia pulita) e lui rimane Host.
- **Sconfitto:** Torna in lobby. Se vuole rigiocare, deve cercare una nuova partita (o creare la propria, separandosi dal vincitore).
- **Pareggio:** Entrambi possono scegliere di creare una nuova partita. Non c'è un "rematch" automatico diretto, ma una scissione in due potenziali nuove stanze.

5. Dettagli Tecnici Client (CLI)

Il client è un'applicazione a thread singolo che utilizza `select()` per multiplexare l'input utente (`STDIN_FILENO`) e la rete.

- **Loop Principale:** Gestisce la navigazione tra stati logici (Lobby -> Hosting -> Waiting -> Playing).
- **Interfaccia:** Utilizza metodi come `clear_screen()` e `print_board()` per ridisegnare l'interfaccia a ogni aggiornamento (nuova mossa, cambio stato lobby), offrendo un feedback visivo pulito nonostante la natura testuale.

6. Esecuzione

Vedere il file `README.md` per le istruzioni relative all'esecuzione del progetto.